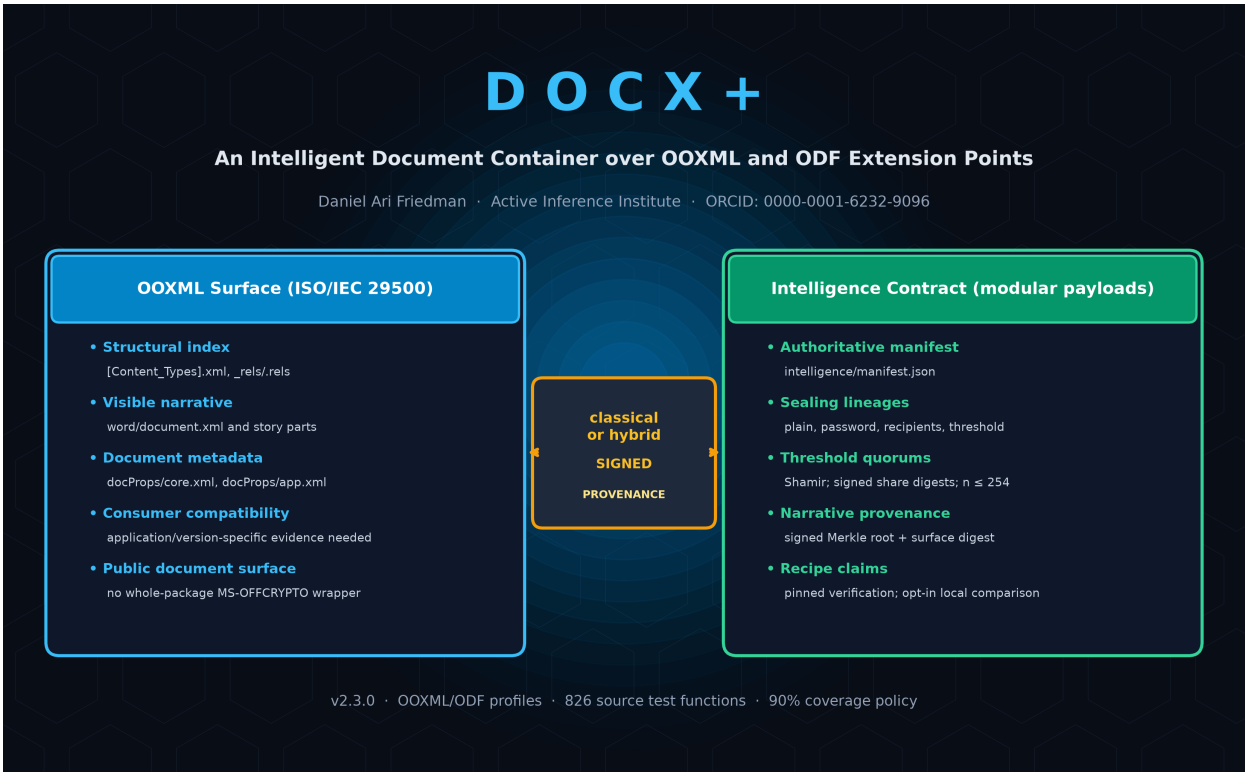



# docxplus: An Intelligent Document Container

*Carrying Signed, Encrypted, Modular Payloads Inside Ordinary DOCX and ODT Files*

**Daniel Ari Friedman**

Active Inference Institute

`daniel@activeinference.institute` · ORCID: 0000-0001-6232-9096

Version: 2.3.0 · Date: 2026-11-10

Standard Reference: ISO/IEC 29500-2 (OPC) · OASIS OpenDocument v1.3/v1.4

Licensed under MIT · Coverage policy: 90% · fresh qualification requires source-bound evidence

# Abstract

A `.docx` file is a ZIP archive governed by the Open Packaging Conventions (OPC; ISO/IEC 29500-2 / ECMA-376-2) (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021), and its OpenDocument counterpart (OASIS ODF v1.3/v1.4) (Brauer, Durusau, and Oppermann 2021) is built the same way. Both standards document extension points — auxiliary package parts, custom XML datastores, application property sets, embedded media, markup-compatibility choice blocks — that exist precisely so a package can carry more than a conforming consumer knows how to read. **docxplus** is an open specification and reference implementation that takes those affordances seriously. A single archive is designed to carry two contracts: an ordinary public document surface and a modular computational payload layer indexed by a signed manifest. Structural checks support selected package invariants; they do not establish complete schema conformance or universal acceptance by Word, LibreOffice or Google Docs.

The design turns on one decision: seal payloads, never the package. Whole-package MS-OFFCRYPTO encryption (Microsoft Corporation 2024) buys confidentiality by destroying the artefact — the prose becomes unreadable to anyone without a credential, and unreadable to archives permanently. Sealing each module instead leaves the surface document public and openable while confidentiality applies exactly where it is wanted. Modules seal under memory-hard password derivation (Argon2id (Biryukov et al. 2021) or Scrypt (Percival and Josefsson 2016), feeding AES-256-GCM), classical X25519 or genuine X25519 + ML-KEM-768 multi-recipient encapsulation (Langley, Hamburg, and Turner 2016; National Institute of Standards and Technology 2024b), Shamir $k$-of-$n$ sharing (Shamir 1979) with signed full-share digests, or two common-policy padded password frames. The standalone share checksum is not polynomial-verifiable secret sharing. Typed payloads include selected `project` trees, but carrying code and data does not by itself reproduce a scientific result.

Provenance rests on a signed Merkle tree (Merkle 1987) over the module roster and a profile-separated, typed/count/length/hash commitment to package parts, type/ODF authority and relationship edges including IDs. Intelligence-manifest bytes are excluded; physical index bytes are represented by their normalized authority. Ed25519 or genuine Ed25519 + ML-DSA-65 signing authenticates the primary algorithm and declared co-signer identities/algorithms (Josefsson and Liusvaara 2017; National Institute of Standards and Technology 2024a). Component standards do not certify the product. Inclusion proofs support module membership; co-signers can express an author-and-institution policy. Authenticity requires the caller to pin the key they trust; the key travelling inside the manifest is self-asserted, and we say so wherever the distinction bites. A `project` module may additionally carry a signed **reproduction attestation** binding its source to an output digest, which a reader checks cryptographically while executing nothing, or re-runs by explicit opt-in under available confinement. Missing confinement refuses execution; readable host files remain exposed. A pinned signed transparency head authenticates inclusion of the complete claim, not independent execution, trustworthy time, or pre-compromise ordering. The output-tree-v3 digest scheme is a clean cutover: missing/old schemes refuse before execution, and local `match` requires both `source_match` and selected-output equality. Linux supplies private PID/proc/session custody. macOS requests fork denial rather than private PID/proc isolation; recipes needing subprocesses require Linux, and macOS enforcement still needs platform evidence.

The OOXML profile provides 5 spec-sanctioned transport channels: custom XML parts, auxiliary package parts, custom document properties, Markup Compatibility and Extensibility (MCE) choice blocks, and least-significant-bit steganography in a carrier image the document visibly displays. A chi-squared heuristic (Westfeld and Pfitzmann 2000) and prefix sweep inspect selected carriers; neither calibrated detection probabilities nor general false-positive/sensitivity claims are established. The source inventory contains 826 test functions under a 90% coverage policy; these are not executed-pass counts. Historical reports and 14 recorded review cycles with 88 confirmed findings describe selected fixtures and past evidence, not fresh software-release qualification. The living manuscript demonstrates a dependency-free source hash, not whole-project scientific reproduction.


# 1 The docxplus Format

## 1.1 What a Document Cannot Currently Do

A `.docx` file is an Open Packaging Conventions (OPC; ISO/IEC 29500-2 / ECMA-376-2) container (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021): a ZIP archive whose mandatory index parts are `[Content_Types].xml` and `_rels/.rels`, and in which every valid part is reached by following an explicit, typed relationship. The standard provides that "a package

may contain additional files", and that mapped content-control data may live in custom XML parts. These are not loopholes discovered by inspection. They are documented extension points, put there so that a package can outlive the assumptions of any one consumer.

Yet the moment a document needs to carry something more — an analysis script, a dataset, a sealed attachment for one named reader — the available options are both bad. Whole-package encryption (the MS-OFFCRYPTO compound envelope (Microsoft Corporation 2024)) protects the payload by making the entire artefact unreadable: the non-secret prose disappears behind a credential, and the file stops being a document at all for anyone without one, archives included. The alternative, an unencrypted attachment sitting beside the file, has no authenticated provenance, no typing, and no access control worth the name. Neither option is a document that carries intelligence; one is a locked box, the other a paper clip.

The gap matters because the document is where results are actually communicated. Reproducible-research practice has converged on the principle that code, data, and the narrative describing them should travel together and be independently re-executable (Peng 2011; Sandve et al. 2013), and the FAIR principles make machine-actionable metadata a first-class requirement rather than a courtesy (Wilkinson et al. 2016). Yet the artefact a reader receives is almost always the one component that carries none of it: a rendered document, severed from its inputs, linked at best to a repository that may move or change. Supplementary-material archives and data repositories address the storage problem while leaving the binding problem open — nothing cryptographically ties the figure on the page to the code that produced it.

This paper asks how structured, authenticated, selectively sealed material can travel through documented Office-package extension points while retaining the intended public document surface. The answer is a conditional construction, not a universal conformance or application-interoperability theorem.

## 1.2 Two Contracts, One Archive

**Definition 1 (Surface Contract).** A package $\mathcal{P}$ satisfies the intended **Surface Contract** when its selected OOXML or ODF profile remains structurally well formed, its required indexes and references resolve, and its document markup meets that profile's rules. Deterministic serialization is a project policy, separate from standards conformance. The implementation's structural validator checks a subset of these obligations, not all schema/application behavior.

**Definition 2 (Intelligence Contract).** A package $\mathcal{P}$ satisfies the **Intelligence Contract** if it contains an authoritative manifest part $\mathcal{M}$ = `intelligence/manifest.json` that maps a finite set of module slots $\mathcal{S} = \{s_1, \ldots, s_k\}$ to their transport channel, content type, Blake2b ciphertext digest, sealing parameters, and optional reproduction claim, signed using Ed25519 (Josefsson and Liusvaara 2017) or genuine dual Ed25519 + ML-DSA-65 (National Institute of Standards and Technology 2024a). Canonical JSON authenticates the primary scheme and declared co-signers.

**Proposition 1 (Conditional Surface Preservation).** Suppose a package satisfies Definition 1, and a channel update preserves required indexes, relationship/manifest authority, profile markup order and the consumer's extension-handling assumptions. Then sealing a payload changes its stored bytes without requiring whole-package encryption. Preservation of surface structure still depends on those channel-specific obligations; arbitrary module changes need not preserve validity, layout, or signatures.

Proposition 1 is a construction argument under stated assumptions, not a proof about every consumer. Fig. 1 locates the parts each channel changes so those obligations can be inspected.

## 1.3 Transport Channels

The OOXML profile defines 5 spec-sanctioned transport channels, each implementing a uniform `embed/extract/capacity` lifecycle:

**Definition 3 (Transport Channel).** A transport channel $\Gamma = (\text{embed}, \text{extract}, \text{capacity})$ is a triple of deterministic operations mapping an OPC package $\mathcal{P}$ and payload bytes $b \in \{0,1\}^*$ to an updated package $\mathcal{P}'$ and a manifest locator record, such that $\text{extract}(\mathcal{P}', \text{embed}(\mathcal{P}, b)) = b$.

Each channel below satisfies Definition 3; the round-trip identity in that definition is what every channel's test asserts against real packages rather than mocks, so the definition is a checked obligation rather than a description. The supported channels comprise:

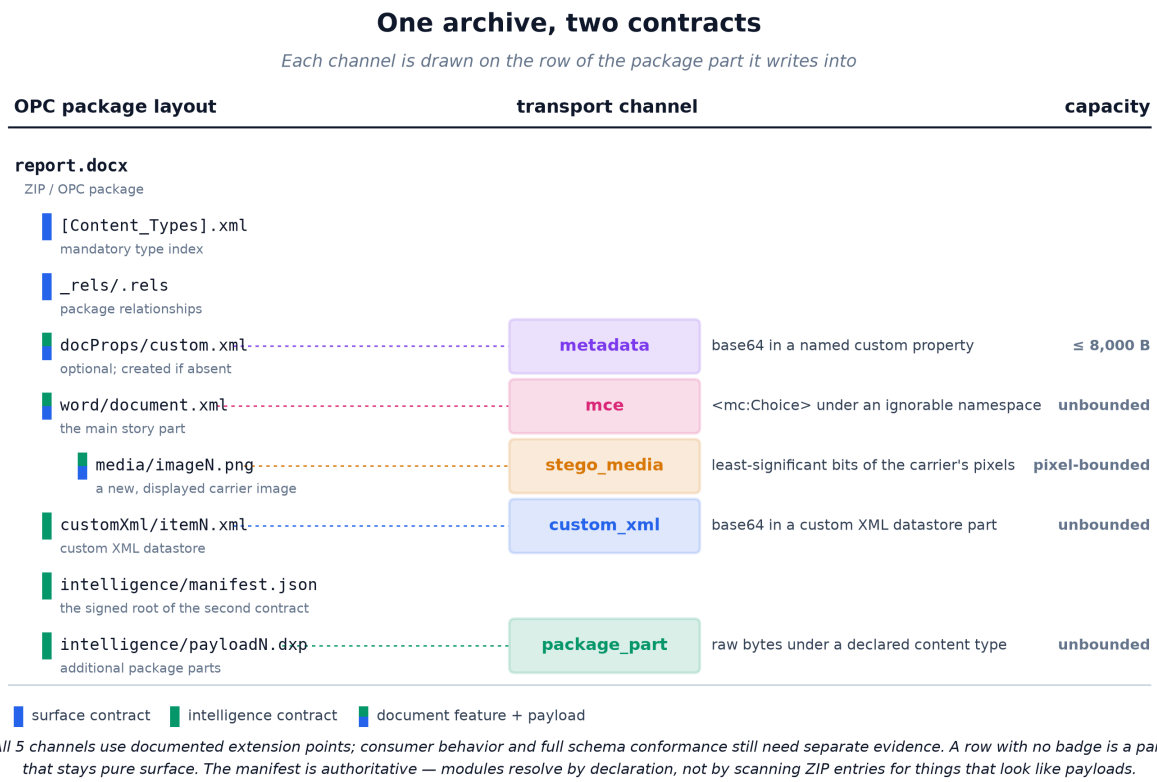



**Figure 1.** The OOXML package tree with every transport channel drawn on the row of the part it writes into. Blue identifies document-surface features, green the intelligence layer, and split markers document features that also carry payloads. Custom properties are optional and created when absent; the media channel adds a displayed image part. These markers do not imply required or pre-existing parts. Capacities are read from live code constants.

- **`custom_xml`** — Base64 payloads in `customXml/itemN.xml` parts registered via `customXml/itemPropsN.xml`. The writer registers datastore metadata through the documented OOXML extension (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021). Checks cover selected structural invariants; neither complete schema conformance nor identical consumer handling is established.
- **`package_part`** — Raw binary payloads in `intelligence/payloadN.dxp` under a declared content type. The high-throughput channel: large datasets, code archives, and ciphertext envelopes.
- **`metadata`** — String-encoded payloads in custom document properties within `docProps/custom.xml`; the ODF profile carries the same bounded contract as `odt_metadata` in `meta.xml` user-defined fields. Bounded at 8000 bytes, which suits routing tags, identifiers, and key shares rather than content.
- **`stego_media`** — Payloads in the least-significant bits of RGB image parts such as `word/media/imageN.png`. The carrier is a figure the document actually displays, so the channel offers concealment on top of confidentiality — with the caveat, quantified in sec. 2.4, that concealment is detectable.
- **`mce`** — Markup Compatibility and Extensibility (ISO/IEC 29500-3) `<mc:AlternateContent>` blocks inside `word/document.xml`. Payloads sit in `<mc:Choice>` guarded by an ignorable namespace, above an *empty* `<mc:Fallback/>`. Avoiding visible fallback content is one channel-specific obligation in Proposition 1, not a guarantee that all consumers preserve every extension.

One rule governs discovery: the manifest is authoritative. Readers resolve modules through manifest declarations, never by walking ZIP entries and guessing, which is what keeps an attacker from introducing a part the reader will treat as intelligence.

### 1.4 The OpenDocument Sibling

Nothing in the design depends on OOXML specifically, and the ODT profile (OASIS ODF v1.3/v1.4 Part 2) (Brauer, Durusau, and Oppermann 2021) demonstrates that. An ODT package places the `mimetype` entry first and stored uncompressed so consumers can identify the format positionally, registers its parts in `META-INF/manifest.xml`, and carries the same signed intelligence layer as the OOXML profile: the manifest of Definition 2, every sealing lineage, the Merkle root over the module set, a surface digest over all non-intelligence-manifest entries and supported ODF authority, and the same co-signature policy.

That parity is a property of the *code*, not a parallel implementation that resembles it. Sealing and unsealing are shared between the two profiles rather than written twice, because a second implementation is free to drift on exactly the security-relevant details: common bounded KDF/power-of-two padding for two password frames (not universal deniability), slot AAD, and signed full-share digest commitments. Two profiles that agree only by inspection eventually disagree under attack.

Untrusted intake is scanned on both sides too, though not by the same code: ODF's threat surface is Basic and Scripts containers and off-package `xlink:href` targets, not VBA and `altChunk`. Running the OOXML scan against an ODF package would pass vacuously, which is worse than not scanning, because it would report clean.

Two channels do not cross over, and saying which is part of the claim. ODF has no custom XML datastore part and no Markup Compatibility element, so `custom_xml` and `mce` are OOXML-only. ODT offers package-entry payloads within archive budgets, visible PNG `stego_media` carriers, and the metadata contract as `odt_metadata` user-defined fields in `meta.xml`. The emitted ODF surface targets the version declared by the builder, not every ODF edition.

The sibling profile is a second front door into the same container, which is why it enforces the same intake ceilings as the OPC reader — entry count, decompression ratio, and rejection of traversal or absolute entry names. A second entrance that is easier to force is not a portability feature; it is the weakest link, and sec. 3.6 records the cycles in which this one was found wanting, twice.

## 2 Implementation

### 2.1 The Package Layer and Its Determinism

The container engine (`opc.py`) implements the Open Packaging Conventions directly over the standard library's ZIP facilities: part indexing, extension-based `Default` and part-specific `Override` content types, and root-anchored relationship graphs. Serialisation is deterministic so that a build is a function of its inputs rather than of the machine that ran it. Every archive entry carries the fixed DOS timestamp 1980-01-01, `[Content_Types].xml` is written first, and the remaining parts follow in lexicographic order. Packaging invariants are enforced in both directions: duplicate ZIP entries and colliding part paths are rejected on read and on write, content types are validated, and XML is parsed with entity expansion disabled.

`wordml.py` synthesises the minimal conforming WordprocessingML surface document. `odt.py` builds the OASIS OpenDocument sibling, placing `mimetype` uncompressed as the first entry, generating `META-INF/manifest.xml`, and applying the same entry-count, decompression-ratio, and path-traversal guards the OPC reader applies. Both writers share reader-compatible per-entry compression policy and enforce package budgets before emission; ZIP directory preflight bounds allocation before rich entry parsing.

### 2.2 Sealing and Key Derivation

**Definition 4 (DXE1 Symmetric Sealing Envelope).** A `DXE1` envelope encrypts payload $P$ under symmetric key $K = \mathrm{KDF}(\mathrm{passphrase}, \mathrm{salt})$ using AES-256-GCM. The module slot name $s$ is bound as Additional Authenticated Data (AAD):

$$C, T = \text{AES-GCM-Encrypt}_K(\mathrm{IV}, P, \mathrm{AAD} = s)$$

The manifest digests the complete stored envelope rather than the plaintext.

**Proposition 2 (Slot-Splicing Invariance).** Assuming AEAD integrity, Definition 4 binds a ciphertext generated for $s_i$ to that AAD; moving it to distinct slot $s_j$ fails verification except with negligible successful-forgery probability.

Binding the slot as AAD gives the conditional integrity argument in Proposition 2. Digesting the stored envelope rather than plaintext removes a direct public confirmation oracle — a plaintext digest would let anyone holding the file test guesses against a short secret without ever attempting decryption. The digest is checkable without any credential, and is checked before decryption is attempted.

Key derivation favours memory-hard functions, which is what raises the cost of offline dictionary attacks on GPU and ASIC hardware:

- **Argon2id** (RFC 9106 recommended profile) (Biryukov et al. 2021) — 64 MiB memory cost, time cost $t = 3$, parallelism $p = 4$.
- **Scrypt** (the default) (Percival and Josefsson 2016) — $N = 32768$ ($2^{15}$), $r = 8$, $p = 1$: the default memory-hard profile, with environment-dependent interactive cost.
- **PBKDF2-HMAC-SHA512** (compatibility) — 600000 iterations, per OWASP 2023 guidance. Algorithm availability does not establish a FIPS-certified product or environment.

Because the envelope declares its own work factors, those factors are attacker-controlled input, and readers cap them before either password frame invokes a KDF. Scrypt bounds both $128 \cdot N \cdot r$ memory and $N \cdot r \cdot p$ work; chaff uses the selected KDF. These declared-work ceilings are not an aggregate process-memory guarantee. sec. 3.6 records why bounding one parameter was insufficient.

Fig. 2 traces a payload through the whole path. It branches only at the sealing mode, and every branch reconverges on the same stored bytes — which is what the manifest records a digest of. The alternative, digesting the plaintext, would ship an offline oracle against the passphrase inside the file the attacker already has. Every parameter in the figure is read from the live constants rather than transcribed.

The recipients lineage keeps identities out of the envelope and, by default, does not keep their number out: one wrapped content key per recipient, plus an explicit count field, so the total is readable from the envelope's length. The manifest records neither identities nor number, which makes the omission an inconsistency rather than a policy — for the blind-review packet this lineage is meant to serve, "sealed to three people" is itself information about the review. Padding raises the slot count to a fixed bucket by wrapping to freshly generated public keys whose private halves are discarded before the call returns, so a padded slot is a real wrap that nobody can open, exactly as the chaff frame is. It is opt-in, and the unpadded default therefore still leaks the count.

Beyond symmetric sealing, both document profiles support classical X25519 recipients (`DXE2`) (Langley, Hamburg, and Turner 2016) or homogeneous genuine X25519 + ML-KEM-768 recipients (`DXE3`) (National Institute of Standards and Technology 2024b). Manifest, co-signature and log signing support Ed25519 (Josefsson and Liusvaara 2017) or dual Ed25519 + ML-DSA-65 (National Institute of Standards and Technology 2024a) through `cryptography` 50 or newer. Both signatures are required under a pin of the entire composite identity. Portable `DXHK` private keys distinguish recipient/signing suites and validate exact lengths and derived public halves. Mixed/ambiguous families and simulated material refuse, with no classical fallback. Component standards do not certify this protocol or product; migrate by fresh keys and full-key pin distribution.

`shamir.py` implements $(k, n)$ sharing over GF(256) (Shamir 1979). Its public wire checksum can be recomputed by an attacker and is not Feldman/polynomial VSS. Threshold records instead commit in the signed manifest to digests of complete issued shares, checked before combination. Trust in those issued bytes requires the pinned manifest signer; polynomial consistency of an honest setup is an assumption. Password modules admit exactly two complete bounded frames before KDF work, then attempt both. Ordinary and decoy frames share KDF parameters and equal power-of-two padding buckets. Size ranges still leak; timing/distributional indistinguishability, coercion resistance and universal deniability are not established.

## 2.3 Binding the Payloads to the Prose

**Definition 5 (Composite Surface Digest).** Let $\mathcal{P}$ be a package and let $\mathcal{M}$ be its intelligence manifest part. The **composite surface digest** $D_{\text{surface}}$ is a domain-separated, counted and length-framed BLAKE2b commitment to a normalized package model. Part records bind names, lengths and SHA-256 content hashes. OPC also binds all content-type declarations and each relationship's source, ID, type, target and mode. ODF uses a distinct profile domain and commits to non-$\mathcal{M}$ content plus supported root/file-entry authority metadata. Intelligence-manifest bytes are excluded; physical index bytes are represented by normalized authority. Unsupported authority refuses.

**Proposition 3 (Conditional Package-Graph Integrity).** Assuming collision resistance of both the content

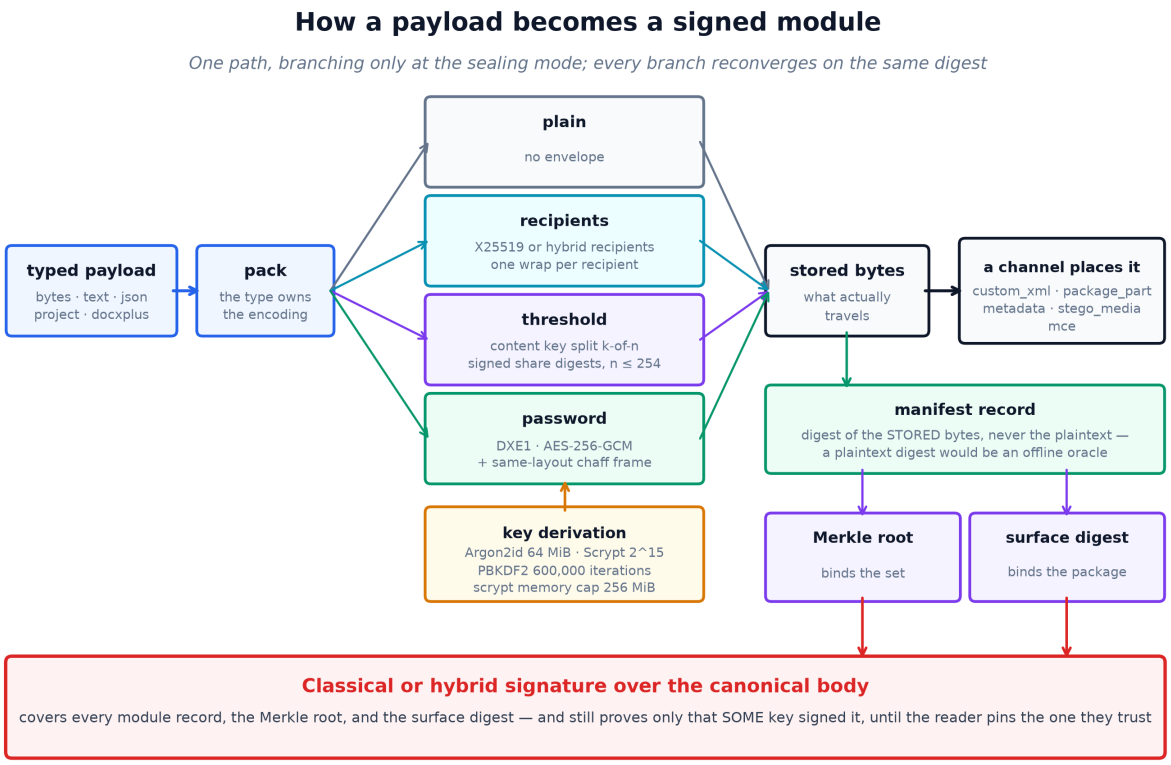



**Figure 2.** How a typed payload becomes a signed module. The path branches only at the sealing mode, and all 4 branches reconverge on the same stored bytes — which is what the manifest digests. Digesting the plaintext instead would leave an offline oracle against the passphrase in a file the attacker already holds. Key-derivation profiles and the ceilings imposed on attacker-declared work factors are read from the live constants rather than transcribed.

and transcript hashes, signature unforgeability and a trusted pinned signer, a change to a committed package-model value in Definition 5 is detected when the surface commitment and signature are recomputed. This does not prove full schema conformance or rendering equivalence.

`payloads.py` retains the typed registry (`bytes`, `text`, `json`, `project`, `docxplus`). Shared stdlib `archive_limits.py` preflights ZIP-directory allocation and bounded release-evidence gzip/TAR metadata. Project capture uses retained descriptors and bounded reads, not a global cross-file snapshot. Project intake bounds compressed/file bytes, members/nodes, physical TAR/PAX metadata and path depth before publishing a fresh private root into an absent/empty destination; unsafe paths/types and missing safe descriptor capabilities refuse. These budgets are not a whole-process-memory quota.

`provenance.py` uses RFC 6962 tree splitting (Laurie, Langley, and Kasper 2013) over module digests. Inclusion proofs contain a logarithmic number of sibling hashes; proof construction and whole-document verification are not log-time claims. The signature covers that root together with the composite surface digest of Definition 5, so tampering with a header, a style, a font table, or the officeDocument relationship breaks provenance exactly as tampering with a payload does. Proposition 3 states the property. An earlier version hashed a list of story-part *names*; sec. 3.6 records why a naming convention cannot carry it. Authenticity, throughout, requires the caller to pin `expected_public_key`; a key that travels inside the manifest is an identity claim, not a verified one. The canonical body also binds the actual primary signing scheme and all declared co-signer identities/ schemes. Strict key framing chooses the verifier; an untrusted label cannot downgrade a hybrid signature.

`transparency.py` maintains an append-only log of reproduction attestations, following the tamper-evident-logging construction of Crosby and Wallach (Crosby and Wallach 2009) and the signed-tree-head discipline that Certificate Transparency established for the same problem in the WebPKI (Laurie, Langley, and Kasper 2013). Hash chaining as a tamper-evidence primitive dates to Lamport's one-way password chains (Lamport 1981); the

lesson the later work adds is that a chain must be *anchored* to mean anything. Chain verification establishes only self-consistency, which is weaker than it sounds: an adversary who rewrites the log from its first entry produces an equally self-consistent chain, and because no entry references the tip, the final entry's body can be edited in place without breaking any linkage. Authenticity therefore rests on a **signed tree head** — an Ed25519 or genuine dual Ed25519 + ML-DSA-65 signature, domain-separated from manifest signing, over the tuple (log length, Merkle root, claimed timestamp). A retained expected head is needed for freshness. `attestation_digest` commits to the complete canonical claim, not only its output digest. Pinned-head inclusion authenticates that claim; it does not witness execution, trustworthy time, or pre-compromise document signing. An offered log without an authenticated head remains unauthenticated.

Deterministic selected outputs make repeated digest comparison useful (Reproducible Builds Project 2024), but do not establish scientific validity. `reproduce.py` runs carried recipes only after explicit opt-in, refusing when the required platform confinement is absent. The signed output-tree-v3 scheme refuses legacy/missing schemes before execution. Local `match` requires `source_match` and output digest equality; toolchain equality is reported separately. Linux private PID/proc/network namespaces and a pinned namespace-init pidfd provide detached-descendant custody: cleanup waits for init exit readiness before monitor reaping or collection, and a failed bounded join refuses collection. macOS requests process-fork denial, not private PID/proc custody; recipes needing subprocesses require Linux. Native macOS enforcement remains unqualified here. Per-process limits are not aggregate quotas. Private scratch does not hide the readable host root or credentials. Use disposable isolation for untrusted code; the living example does not execute the full pipeline.

## 2.4 Concealment, and Measuring It

`lsb.py` provides a pure-Python RGB PNG codec. `DXL1` has a four-byte magic, uint32 length and exact payload; truncated headers, over-capacity lengths and uint32 overflow refuse rather than silently slicing. `steg_bridge.py` uses the actual Rust `capacity` command and carrier dimensions, with identical stable `payload.bin` filename, MIME metadata and signing policy in capacity and encode. Default packets are unsigned/non-ECC; signing requires an explicit key and identity pinning is separate. No ECC knob is exposed.

Discovery uses explicit `STEGANOGRAPHER_BIN` or PATH, never a sibling search/Cargo build; invalid explicit configuration fails. RGB processing has a shared pixel budget, not a total-memory guarantee.

Pearson adjacent-pair equality statistics report `p_value`, an upper-tail reference probability, not an embedding posterior. Each populated pair contributes a degree of freedom; insufficient samples/expected counts return unavailable, not a clean negative. Balanced untouched covers can flag. Prefix sweeps reuse samples and are not independently calibrated. Selected fixtures do not establish general sensitivity, false-positive rates or a payload-size estimator. `redundancy=N` needs a surviving digest-matching replica; it does not promise that edited packages retain valid provenance.

`mce.py` implements the Markup Compatibility and Extensibility channel (ISO/IEC 29500-3) (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021), wrapping payloads in `<mc:Choice>` elements under an ignorable namespace in `word/document.xml`. A compliant application that does not recognise the namespace discards the Choice branch and renders the fallback, with no warning and no error. That fallback is deliberately empty, so a concealed module leaves the paragraph count unchanged; an earlier version emitted a blank `<w:p>` and thereby falsified the independence it was supposed to demonstrate. Placement is equally load-bearing: `CT_Body` is `(EG_BlockLevelElts*, sectPr?)`, so the block is inserted *before* the body-level `<w:sectPr>` rather than appended at `</w:body>`.

## 2.5 Composition, Validation, and Untrusted Input

`container.py` runs the document lifecycle from both ends. `DocxPlusBuilder` queues typed modules, applies per-module sealing, computes the surface digest and Merkle root, and serialises deterministically; `DocxPlusReader` recovers the manifest, extracts payloads, and verifies provenance, co-signers, and attestations.

`odt_container.py` provides `OdtPlusBuilder` and `OdtPlusReader`, the same lifecycle over an ODF package. It reuses `container`'s sealing step and unsealing path verbatim; only placement differs, since ODF locates parts through `META-INF/manifest.xml` rather than a relationship graph.

`validate.py` checks selected structural and intelligence invariants in both profiles: graph reachability, namespaces, collisions, digest/Merkle/surface consistency, and signatures if present. It is not a complete ISO/XSD/RNG

validator or an Office implementation proof. Signature self-consistency remains distinct from pinned authenticity; an unsigned document can pass structural checks without provenance. Intake reports XML parse failures as non-clean rather than silently treating unscannable parts as safe.

The shared non-decoding inspection path retains original manifest evidence, including the serialized Merkle declaration, rather than rebuilding away a discrepancy. Batch audit preserves each input's structural, threat and identity findings independently. Manifest-aware comparison reports logical package, module and signer-policy differences without inferring plaintext or rendered equivalence. Bulk export recovers exact packed bytes to a fresh private directory with atomic no-replace publication; project and nested-document payloads remain inert. These workflows reuse complete key pins and shared reader unsealing, and do not turn successful decryption or unpinned self-consistency into authenticity.

One validator rule exists for a feature that does not: a package carrying whole-package OPC signatures is rejected unless their combined reference set covers the intelligence manifest and every part it names. A signature enumerating only the conventional Word parts would render as valid in a desktop office suite over a document whose intelligence layer had been stripped, so the trust indicator would be attesting the absence of what a reader assumes it covers. Writing the rule before the signing code is deliberate; it is the invariant that keeps the feature from becoming a way to launder missing payloads (sec. 4.3).

`intake.py` is the hardened gateway for files of unknown origin, reporting external relationship targets, macro parts, and foreign `altChunk` imports statically, executing nothing. The `docxplus` CLI is a thin orchestration layer over these modules and implements no logic of its own; it currently exposes 22 commands: analyze-carrier, audit, build, compare, export, extract, graph, inspect, keygen, odt-build, odt-extract, odt-inspect, odt-scan, odt-validate, reproduce, scan, transparency-append, unpack-project, validate, verify, verify-reproduction, verify-transparency.

# 3 Evaluation

> Live constants, source inventories and available reports feed the manuscript variable extractor. Token hydration prevents literal drift, not stale evidence. Report-derived values below are historical unless a source-bound release receipt identifies the exact final inputs and successful run. No fresh full-suite, coverage, native-Office or render result is asserted here.

## 3.1 Round-Trip Integrity Across Every Sealing Mode

The reference dossier (`src/docxplus/reference_docs.py`, `scripts/04_dossier.py`) carries 5 heterogeneous modules in a single archive, covering all 3 sealing lineages (`password`, `recipients`, `threshold`) across 1 of the 5 transport channels (`package_part`). Sealing is what the dossier is for; the remaining channels are exercised by the round-trip and channel suites rather than here. One `ReferenceDossier` supplies serialized artifact bytes and ephemeral credentials to both the exporter and table; checks read those bytes through its reader:

| Slot | Channel | Sealing |
|---|---|---|
| `annex` | package_part | password |
| `notes` | package_part | password |
| `review` | package_part | recipients |
| `source` | package_part | password |
| `vault` | package_part | threshold |

The generated dossier table specifies the fixture's intended extraction/verification checks:

- **Structure.** Selected OPC graph checks (collisions and relationship resolution), not full ISO conformance.
- **Provenance.** Signature, module Merkle root and package-surface commitments recompute; a caller-pinned key is needed to authenticate the signer.
- **Sealing.** Selected password, recipient, threshold and decoy round trips, including signed full-share digest membership before combine. Standalone share checksums do not establish malicious-share resistance.
- **Project extraction.** The dossier source slot is extracted but is not reproduction-attested. Living/synthetic suites own their recipe claims and separate local execution evidence.

The source inventory comprises 826 test functions under a 90% coverage policy, not that many freshly passed cases. Tests use real primitives, files and subprocesses; optional compiled-backend and native-application cases require their capabilities and cannot count a skip as an exercised result.

## 3.2 A Manuscript That Carries Its Own Repository

`scripts/05_living_manuscript.py` packs the selected docxplus repository tree into a `.docx` with a signed attestation for a dependency-free hash of selected Python source files. It does **not** rerun the tests, figures, rendered paper or full scientific workflow. The carried archive is distinct from this rendered PDF.

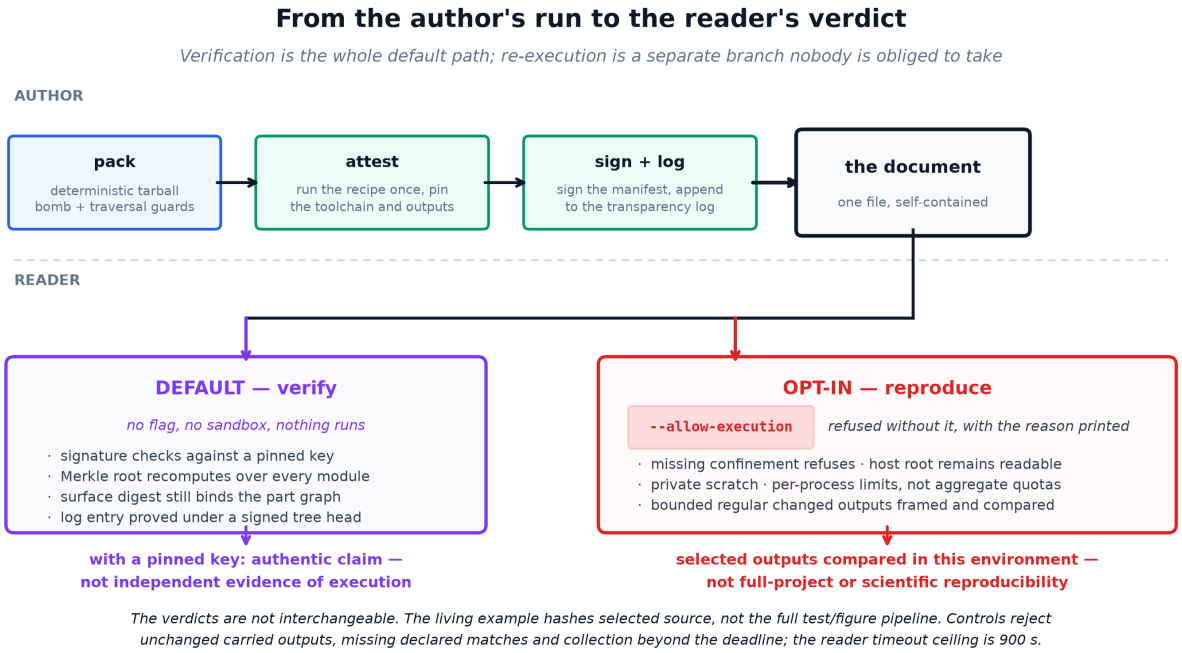



**Figure 3.** Authoring runs left to right and ends at one self-contained file; what a reader does with that file forks. The default branch is entirely cryptographic and executes nothing. The other requires an explicit flag and re-runs the attested command under confinement. The fork exists because the two verdicts are not interchangeable: one says the author's claim has not been altered, the other says it reproduces on this machine, and neither substitutes for the other.

A recipient can engage at three levels of increasing commitment. The last two are the branches that fork in fig. 3:

1. **Read it.** The public document surface is intended for ordinary Office consumers; native compatibility remains an application/version/fixture-specific empirical question.
2. **Verify it, executing nothing.** `docxplus verify-reproduction --expected-key ...` authenticates the signed recipe claim against carried bytes. Unpinned verification establishes only self-consistency; neither path witnesses execution.
3. **Re-run it.** `docxplus reproduce` opts in to instantiating the project in a confined sandbox, executing the attested command, and comparing the resulting digests against the author's.

The third tier is optional and potentially dangerous. A successful local run compares the declared selected outputs in the observed environment; it does not establish independence from readable host inputs, an exact environment match, or scientific validity. The source-hash demonstration is intentionally narrower than a whole-project reproduction claim and is insensitive to changes outside its selected source set.

## 3.3 Carrying a Project Out and Back

The claim that a document can carry the software that produced it is only worth as much as the fidelity of the round trip, so the fidelity is measured rather than asserted. `scripts/06_project_roundtrip.py` builds a project tree chosen to contain exactly the cases naive packing loses — an executable entrypoint, an empty directory, a

zero-byte file, filenames with spaces and non-ASCII characters, two files with identical content, and build junk that must *not* travel — carries it into both containers, then reads actual exported artifacts and enumerates both source and extracted trees. Extra entries fail alongside missing/changed files, execute bits and empty directories.

The available report records a 14-file, 9-directory selected synthetic tree and 18 of 18 checks, with 12 compared files. Those tokens describe the report, not a fresh run. It reports package sizes of 5,390 bytes (DOCX) and 4,552 bytes (ODT), plus cross-profile packed-payload parity. Shared packing makes consistent behavior plausible, but does not replace independent end-to-end evidence for either profile or imply arbitrary-project fidelity.

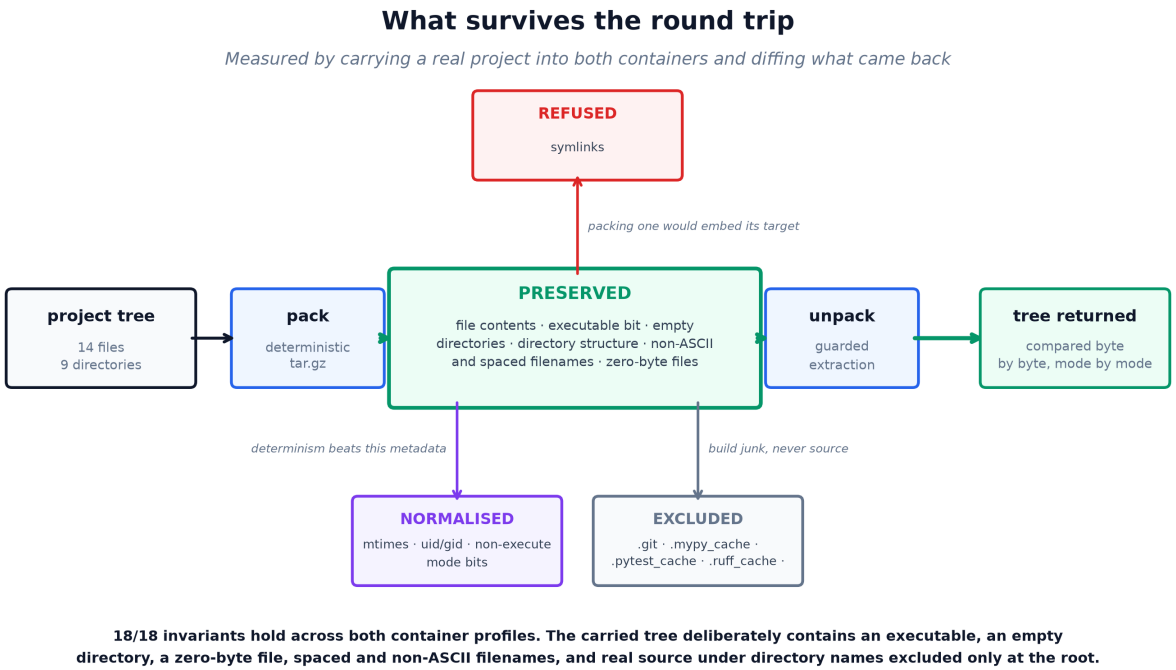



**Figure 4.** A selected project fixture carried into a container and compared with what came back. The preservation contract includes file contents and the executable bit; metadata is normalized, symlinks refused by default and build directories excluded. Counts come from the available harness report at render time, not from a freshness check or a general fidelity proof.

Fig. 4 states the contract in full: what is preserved, what is normalised away on purpose, what is refused outright, and what never travels at all.

Two results are worth separating from the pass count. The first is what the harness found on its way to passing: packing previously forced every file to mode `0644`, so a carried `run.sh` came back non-executable, and empty directories vanished under a files-only walk. Both are now preserved. The second is a security finding rather than a fidelity one. `Path.is_file()` follows symbolic links, so a tree containing `creds -> ~/.ssh/id_rsa` had the *key's contents* packed into the document under the link's name, with nothing in the manifest to suggest it. The unpack side had always rejected link members; the pack side was the open door. Symlinks are now refused unless a caller passes `follow_symlinks=True` and thereby says so in a way that survives review.

Fig. 5 sets the two profiles side by side. What matters there is not the count of ticks but the dividing line: above it the profiles execute the *same code*, so parity is structural rather than a coincidence that survives until someone patches one side.

The harness also exercises what a single-container round trip cannot reach: a signed `.docx` carried as a nested module *inside* a signed `.odt`, opened through a dispatcher that reads the container's own magic rather than trusting the caller to declare it, with the inner document's provenance and project both verifying after extraction.

## 3.4 Channel Capacity

Capacity is channel- and backend-specific; no channel-local limit removes package entry/total/count budgets.

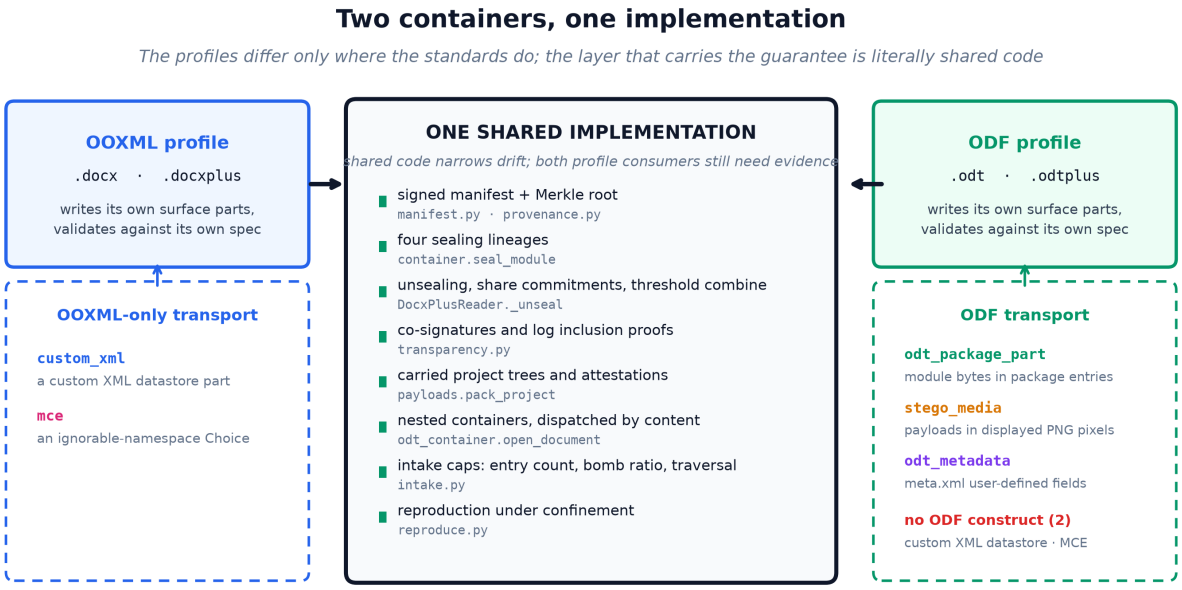



**Figure 5.** The two profiles drawn as shells around a shared core. Shared sealing and provenance code narrows security drift; each profile still needs consumer-specific evidence. ODT implements package parts, displayed media and metadata user fields. The OOXML custom XML datastore and MCE constructs have no ODF-native counterpart. Transport differences do not require a second sealing or signature implementation.

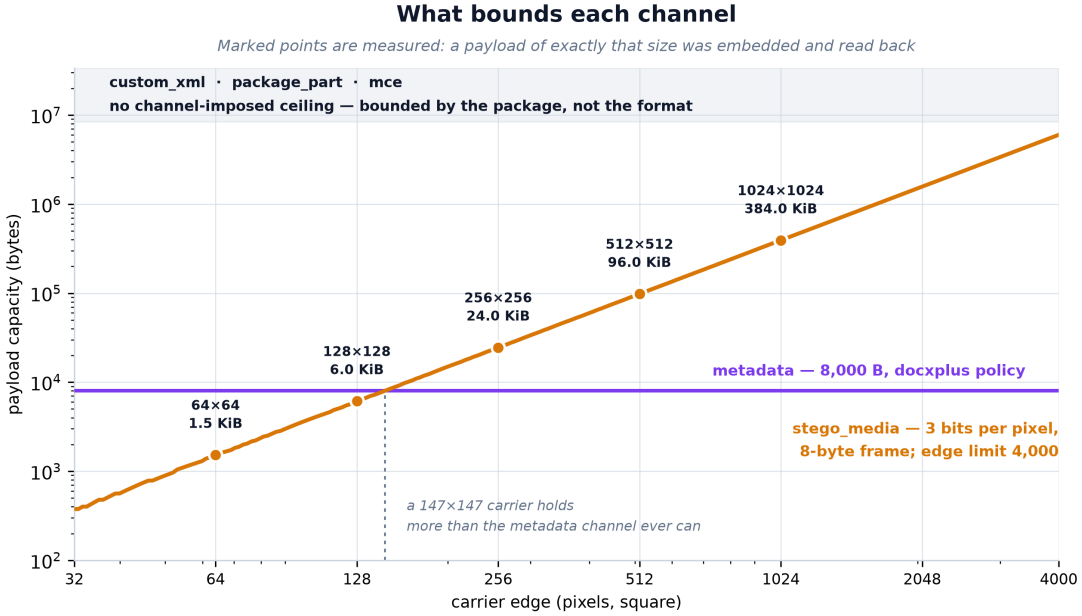



**Figure 6.** Channel capacity under the selected Python LSB framing and package budgets. Metadata has a docxplus-imposed per-payload ceiling; media scales with carrier area only within the decoded-pixel budget; the other channels have no channel-local ceiling. Regeneration performs boundary round trips before plotting measured points. A committed plate without a source-bound receipt is historical, not fresh capacity evidence.

Fig. 6 distinguishes implementation policy, carrier framing/pixel admission and package budgets:

- **`metadata`** has a conservative implementation limit of 8000 payload bytes in `docProps/custom.xml` (and ODF `odt_metadata` fields in `meta.xml`), not a standards-imposed maximum. It is intended for routing headers, identifiers, status flags and key shares.
- **`custom_xml`, `package_part`, and `mce`** have no channel-local ceiling, but XML/base64, part, archive metadata, count and total-size limits still apply.
- **`python_lsb` media** uses DXL1 framing: a $256\times256$ RGB PNG holds 24568 bytes and a $512\times512$ PNG holds 98296. Rust media instead queries actual CLI capacity with the same metadata/signing policy as encode. Replication requires a surviving digest-matching carrier; it does not preserve signed package authenticity through edits.

## 3.5 Conformance, Openability, and Determinism

The construction uses documented OPC/ODF extension points (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021; Brauer, Durusau, and Oppermann 2021) and payload-level rather than whole-package encryption (Microsoft Corporation 2024). Structural checks are not complete schema validation. Native application openability needs receipts naming each application/version and emitted fixture; no current Word/Google Docs proof or fresh LibreOffice conversion is inferred from unit tests or PDF generation. The release Office gate renders actual installed-wheel DOCX/ODT, saves through Writer, then renders again. Its tool-bound receipt retains original/saved document/PDF/text/raster hashes and visible/hidden sentinel checks. It qualifies the recorded fixture's visible rendering only, not payload survival or signature validity after Office editing. Qualification remains conditional until a fresh receipt is imported.

Repeated unencrypted-build tests express deterministic serialization obligations. Fresh salts/nonces make encrypted output intentionally nonidentical; fixed part order alone does not establish whole-workflow determinism.

## 3.6 Adversarial Verification

The audit record describes 14 review cycles and 88 confirmed findings. Historical regression claims, current static findings, observed baseline failures and unavailable fresh qualification are distinguished in `docs/redteam-audit.md`. Counts are not a security proof or pass tally. The newest source changes require final source-bound integration evidence; prior successful runs do not automatically qualify new format framing, keys, consumers or platform controls.

Fig. 7 maps the classes to their controls. Four findings generalise past this implementation, and they are the reason the review is worth reporting at all.

**A naming convention cannot carry a security property.** The manifest signature originally bound a list of part *names*, while OPC resolves the rendered document through the officeDocument *relationship* (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021). Every test passed, because each asserted what the code said it did; the defect lay in the space between two individually correct components, the position Dolev and Yao formalised for protocols (Dolev and Yao 1983). What closed it was a change of question — asking of each constraint whether it was a law of the format or a convention this project had chosen — rather than a better test.

**A control needs an authenticated reference.** A public share checksum can be recomputed after mutation; requiring its header never made it Feldman VSS (Feldman 1987). Signed full-share digest commitments instead authenticate issued bytes under a pinned trusted manifest. That does not prove polynomial consistency or an honest dealer. The distinction is between an accidental-corruption check and an adversarial authenticity boundary; downgrade controls alone cannot upgrade the former into the latter.

**Absence of a failing test is not evidence of a working control.** Two findings were silent: a determinism guarantee implemented by assignment to a non-existent attribute, and a capability documented in prose with no implementing code path. Both produced artefacts that looked correct under inspection. Tests defend against the failure modes they were shaped to anticipate, and neither of these was shaped like a failure — the selection problem Goodenough and Gerhart set out at the foundation of testing theory (Goodenough and Gerhart 1975).

**A guarantee delegated to a neighbour is not a guarantee.** The Merkle construction padded odd levels by duplicating the trailing node, which makes the tree over three leaves hash identically to a four-leaf tree whose

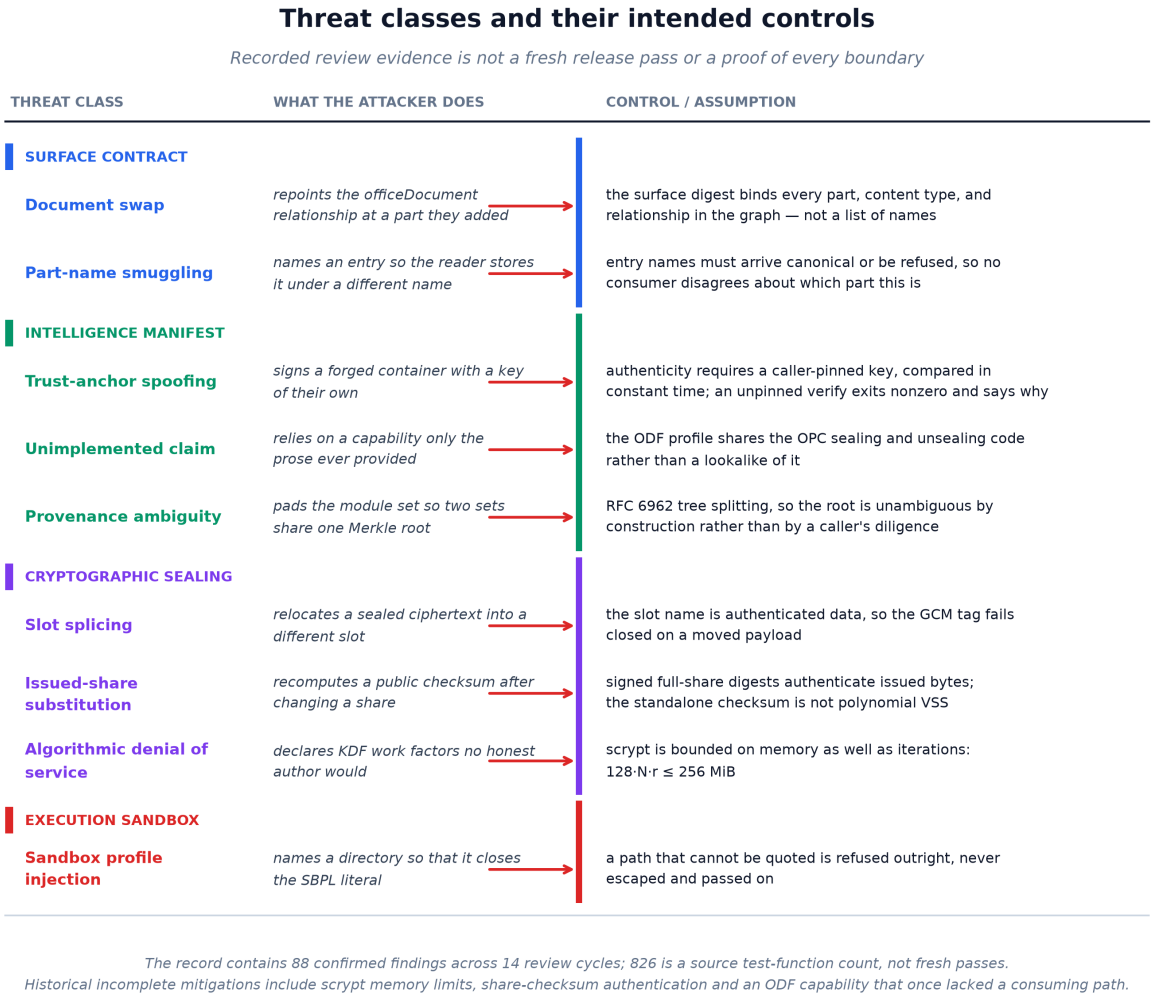



**Figure 7.** Threat classes grouped by layer and the intended controls that address them. Arrows depict design boundaries, not proof that every adversary or current platform probe is blocked. The audit distinguishes observed failures, source-derived mitigations and unavailable qualification evidence.

last two leaves are equal — the second-preimage ambiguity found in Bitcoin as CVE-2012-2459. The documented promise that adding a module always changes the root was therefore false in that case. It was unreachable through a well-formed manifest only because slots are unique, and slots were unique only because a *different* module checked them on the write path; the read path did not. In the same round the validator was found to recompute the Merkle root but not the surface digest, so a package whose visible prose had been rewritten passed `docxplus validate` with no findings at all, protected only by whether the reader happened to run `verify` instead. Both fixes move the check into the component that owns the property: RFC 6962 tree splitting makes the root unambiguous by construction rather than by a caller's diligence, and the validator recomputes what it had been trusting. A property that holds because something else is careful is a property with an undocumented dependency, and undocumented dependencies are what change when code does.

The record also carries a negative result. A sample-pair analysis estimator (Dumitrescu, Wu, and Wang 2003) was implemented and measured against carriers embedded at known rates. It proved monotonic in the true rate but mis-scaled by roughly an order of magnitude and dependent on carrier statistics, so it was withdrawn rather than shipped under a name implying a calibration it did not possess.

# 4 Conclusion

## 4.1 What This Demonstrates

An ordinary Office container can carry authenticated, selectively sealed computational payloads through documented extension points. The mechanism separates a public **surface contract** from a signed modular **intelligence contract**. Preservation depends on profile/channel construction obligations, supported consumer behavior and cryptographic assumptions; universal readability or full conformance is not proved.

Concretely:

1. **Payload-level rather than package-level sealing.** Argon2id (Biryukov et al. 2021), Scrypt (Percival and Josefsson 2016), and AES-256-GCM; X25519 multi-recipient encapsulation (Langley, Hamburg, and Turner 2016); threshold sharing (Shamir 1979) with signed full-share digest commitments; decoy chaffing — all applied to payloads, not the whole archive.
2. **Provenance over the package graph.** Merkle trees (Merkle 1987) over the module set bound together with every part, content type, and relationship — not merely the story parts — under one signature (Josefsson and Liusvaara 2017).
3. **Authenticatable reproduction claims.** Carried project archives and signed attestations can be checked without execution under a pinned key. Local re-execution is separate, opt-in and requires confinement; the living source-hash example is not full-project scientific reproduction.
4. **5 documented OOXML transport channels**, spanning custom XML, package parts, document properties, MCE choice blocks and LSB steganography, with an uncalibrated in-tree inspection heuristic (Westfeld and Pfitzmann 2000) rather than a statistical presence/absence guarantee.
5. **Standards parity** across Office Open XML (*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)* 2021) and OASIS OpenDocument Text (Brauer, Durusau, and Oppermann 2021): the same signed manifest, sealing lineages, and provenance in both containers, implemented by shared code rather than by two implementations that agree today. Parity holds at the intake boundary too, where the sibling profile enforces the same ceilings. Two OOXML channels have no ODF analogue and are named as such.

## 4.2 What This Does Not Do

The boundaries below are load-bearing. Stating them is part of the contribution, since a security property a reader misunderstands is worse than one absent (`docs/security-model.md`).

The boundaries below correspond to the rungs in fig. 8; each one names a conclusion the format does *not* license. Stating them precisely is itself a design obligation: cryptographic software fails far more often through misunderstood interfaces and unstated assumptions than through broken primitives (Lazar et al. 2014), and a system whose guarantees a reader over-reads has failed even when every algorithm in it is sound.

- **The surface is public on purpose.** docxplus is not digital rights management. The prose is meant to be readable, and confidentiality reaches only the modules marked for it.
- **Concealment is obfuscation, not secrecy.** Adjacent-pair `p_value` measures compatibility with an equal-pair reference model, not embedding probability. Sparse samples are unavailable; balanced untouched covers can flag. Prefix sweeps are dependent and no general detection calibration is established.
- **Common decoy policy is not universal deniability.** Exactly two bounded frames share visible KDF parameters and power-of-two padding, whether carrying chaff or a cover story. Buckets leak size ranges; timing/distributional indistinguishability and resistance to compelled disclosure are not proved.
- **Integrity is not authenticity.** A valid signature proves the manifest was signed by whoever holds the embedded key. Only comparison against a caller-pinned key makes that signer anyone in particular.
- **A local comparison is not scientific proof.** It measures selected outputs of a declared run in an observed environment. A signed attestation authenticates a claim, not execution; a pinned transparency head adds authenticated inclusion, not trustworthy time or pre-compromise ordering. Host files remain readable under the confinement wrappers; scrubbed environment variables do not hide credentials.
- **Desktop suites do not verify the intelligence signature.** The signed body now binds the module set and the whole package graph, but Word and LibreOffice will not check it. Native validation is a different signature (OPC XML-DSig), and that is the first item below.

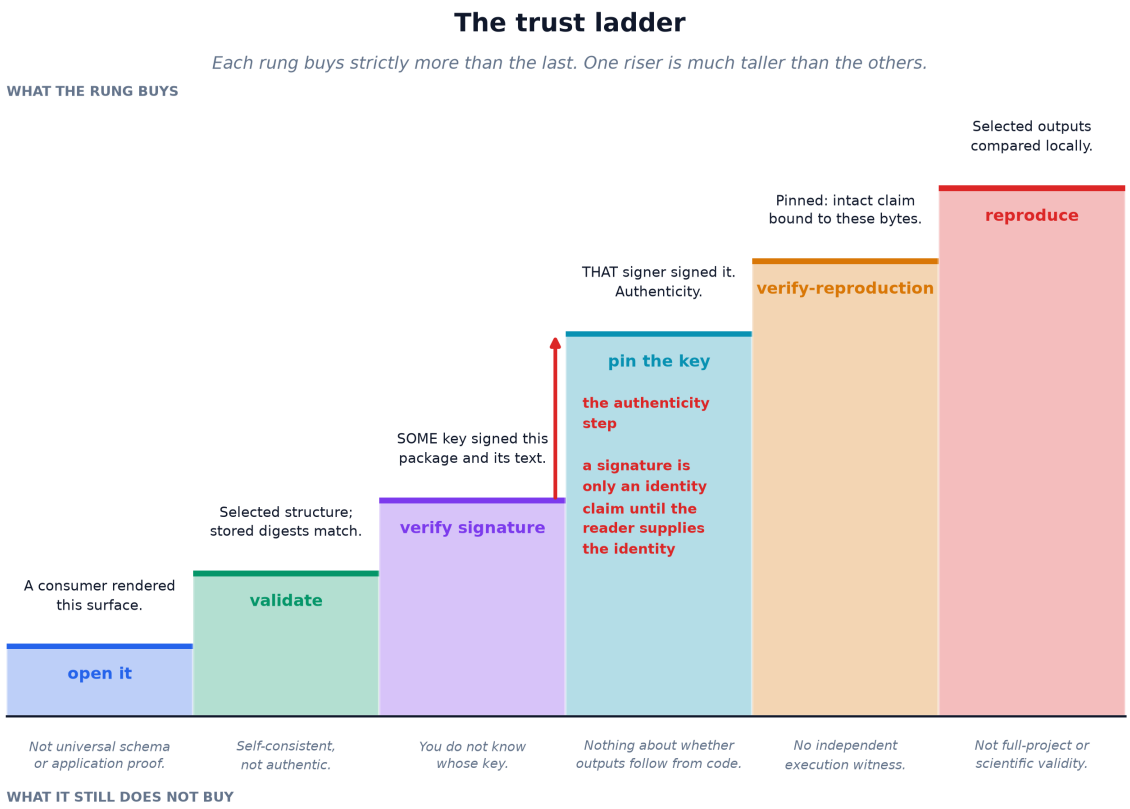



**Figure 8.** What a reader may conclude at each rung of verification, what it costs them, and — the column that usually goes missing — what it still does not buy. The rungs are cumulative and none is skippable. The step from *verify signature* to *pin the key* is the one that carries the weight: until the reader supplies the identity they trust, a valid signature establishes only that the document is internally consistent with itself, which is a property a forger can supply just as easily as an author.

- **An Office save is not payload preservation.** The release gate renders actual installed-wheel DOCX/ODT, saves via Writer, then renders again, retaining PDF/text/raster and tool-bound receipts. It checks visible text and nonblank previews, not survival of modules or signatures through editing. Current qualification remains conditional on the release owner's actual source-bound receipt.

## 4.3 Future Work

The nearest open item is producing a whole-package OPC XML-DSig signature alongside the manifest signature, which would let desktop office suites validate a docxplus document natively. The assessment in `docs/opc-signatures.md` sets out why it is not yet implemented: it needs canonical XML and an X.509 trust story, and Office will not verify Ed25519. The invariant it must satisfy is already enforced. An OPC signature enumerating only the conventional Word parts would display as valid over a package whose intelligence layer had been stripped, which is worse than no signature at all — it would launder the absence of the thing it appeared to attest. The validator therefore already rejects any package whose OPC signature reference set omits a manifest-named part, written before the signing code precisely so the feature cannot ship without it.

Beyond that: calibrating sample-pair analysis against natural-image carriers, so the withdrawn estimator of sec. 3.6 can return with the accuracy its name implies; validating produced documents against Office-o-tron and the ODF Toolkit in continuous integration; establishing formal arguments for container parsing (none are claimed here); and selective attribute disclosure via zero-knowledge proofs, for workflows where a reader must confirm a payload property without opening it.

# 5 Manuscript Status

- **Project:** docxplus
- **Manuscript title:** docxplus: An Intelligent Document Container over OOXML and ODF Package Extension Points
- **Location:** `docs/manuscript/` — resolved by `docxplus.project_paths.source_manuscript_dir()`, so no reader hardcodes the path
- **Type:** Active publication-target manuscript (6 section files)
- **Status file purpose:** Tracks publication-readiness of the manuscript content in this directory, migrated here from the retired root-level `manuscript/` tree.

## 5.1 Evidence status

DOCX and ODT use authenticated format 3.0. Report-derived tokens remain intact, but token hydration is not evidence freshness. Rendered artifacts require a source-bound release receipt identifying the exact successful run. Fresh full-suite/coverage, PDF/render, installed-wheel, real-backend and LibreOffice evidence belongs in those receipts, not this editorial status file. Microsoft Word, Google Docs and macOS confinement remain unqualified without dedicated evidence. The living example hashes selected source; it does not reproduce the full research pipeline. Publication readiness requires qualification receipts and regenerated artifacts, not editorial replacement of metric tokens. The Office render-save-render gate retains tool-bound document/PDF/text/raster receipts; it does not assert payload survival or valid provenance after an Office save. Format-2 containers and older reproduction digests require trusted rebuild/re-attestation, not version/scheme-field rewriting. Output-tree-v3 and genuine portable hybrid workflows need regenerated artifacts and current-source evidence before publication claims.


Biryukov, Alex, Daniel Dinu, Dmitry Khovratovich, and Simon Josefsson. 2021. "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications." Request for Comments. RFC 9106; IETF. https://doi.org/10.17487/RFC9106.

Brauer, Michael, Patrick Durusau, and Lars Oppermann. 2021. *Open Document Format for Office Applications (OpenDocument) Version 1.3*. OASIS.

Crosby, Scott A., and Dan S. Wallach. 2009. "Efficient Data Structures for Tamper-Evident Logging." In *Proceedings of the 18th USENIX Security Symposium*, 317–34.

Dolev, Danny, and Andrew C. Yao. 1983. "On the Security of Public Key Protocols." *IEEE Transactions on Information Theory* 29 (2): 198–208. https://doi.org/10.1109/TIT.1983.1056650.

Dumitrescu, Sorina, Xiaolin Wu, and Zhe Wang. 2003. "Detection of LSB Steganography via Sample Pair Analysis." *IEEE Transactions on Signal Processing* 51 (7): 1995–2007. https://doi.org/10.1109/TSP.2003.812753.

Feldman, Paul. 1987. "A Practical Scheme for Non-Interactive Verifiable Secret Sharing." In *28th Annual Symposium on Foundations of Computer Science (FOCS)*, 427–37. https://doi.org/10.1109/SFCS.1987.4.

Goodenough, John B., and Susan L. Gerhart. 1975. "Toward a Theory of Test Data Selection." *IEEE Transactions on Software Engineering* SE-1 (2): 156–73. https://doi.org/10.1109/TSE.1975.6312836.

*Information Technology – Document Description and Processing Languages – Office Open XML File Formats (Parts 1–4)*. 2021. International Organization for Standardization / International Electrotechnical Commission.

Josefsson, Simon, and Ilari Liusvaara. 2017. "Edwards-Curve Digital Signature Algorithm (EdDSA)." Request for Comments. RFC 8032; IETF. https://doi.org/10.17487/RFC8032.

Lamport, Leslie. 1981. "Password Authentication with Insecure Communication." In *Communications of the ACM*, 24:770–72. 11. https://doi.org/10.1145/358790.358797.

Langley, Adam, Mike Hamburg, and Sean Turner. 2016. "Elliptic Curves for Security." Request for Comments. RFC 7748; IETF. https://doi.org/10.17487/RFC7748.

Laurie, Ben, Adam Langley, and Emilia Kasper. 2013. "Certificate Transparency." RFC 6962. IETF. https://doi.org/10.17487/RFC6962.

Lazar, David, Haogang Chen, Xi Wang, and Nickolai Zeldovich. 2014. "Why Does Cryptographic Software Fail? A Case Study and Open Problems." In *Proceedings of the 5th Asia-Pacific Workshop on Systems (APSys)*. https://doi.org/10.1145/2637166.2637237.

Merkle, Ralph C. 1987. "A Digital Signature Based on a Conventional Encryption Function." In *Advances in Cryptology – CRYPTO '87*, 293:369–78. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/3-540-48184-2_32.

Microsoft Corporation. 2024. "[MS-OFFCRYPTO]: Office Document Cryptography Structure." v20240521. Microsoft Corporation.

National Institute of Standards and Technology. 2024a. "Module-Lattice-Based Digital Signature Standard." FIPS 204. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.FIPS.204.

———. 2024b. "Module-Lattice-Based Key-Encapsulation Mechanism Standard." FIPS 203. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.FIPS.203.

Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27. https://doi.org/10.1126/science.1213847.

Percival, Colin, and Simon Josefsson. 2016. "The Scrypt Password-Based Key Derivation Function." Request for Comments. RFC 7914; IETF. https://doi.org/10.17487/RFC7914.

Reproducible Builds Project. 2024. "Reproducible Builds: A Set of Software Development Practices for Independently Verifiable Paths from Source to Binary Code." https://reproducible-builds.org/.

Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. "Ten Simple Rules for Reproducible Computational Research." *PLOS Computational Biology* 9 (10): e1003285. https://doi.org/10.1371/journal.pcbi.1003285.

Shamir, Adi. 1979. "How to Share a Secret." *Communications of the ACM* 22 (11): 612–13. https://doi.org/10.1145/359168.359176.

Westfeld, Andreas, and Andreas Pfitzmann. 2000. "Attacks on Steganographic Systems: Breaking the Stegovan Camouflage." In *Information Hiding*, 1768:61–76. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/10719724_5.

Wilkinson, Mark D. et al. 2016. "The FAIR Guiding Principles for Scientific Data Management and Stewardship." *Scientific Data* 3: 160018. https://doi.org/10.1038/sdata.2016.18.